



**PROJETO FINAL
BIG-DATA**

GABRIEL ROGÉRIO APARECIDO PAES SILVA
GILSON DOS SANTOS JUNIOR
GUILHERME FABRICIO BRITO DA ROSA
GUSTAVO ALESSANDRO DE SOUZA SABINO

LAVRAS
2026

PROCESSAMENTO DE DADOS MASSIVOS

RELATÓRIO

Relatório apresentado à disciplina de Big-Data, do professor Vinicius Vitor, para obtenção de nota relativa ao período vigente.

LAVRAS
2026

RELATÓRIO

1. Contexto e Motivação

O Counter-Strike: Global Offensive (CS:GO) é um dos jogos de tiro em primeira pessoa mais populares do mundo, reunindo milhões de jogadores e gerando uma enorme quantidade de dados a cada partida. Durante um jogo competitivo são registrados milhares de eventos, como movimentação dos jogadores, utilização de armas, eliminações e eventos de dano.

Plataformas como a ESEA armazenam essas informações em grandes bases de dados, permitindo a realização de análises estatísticas sobre o desempenho dos jogadores e das partidas.

O objetivo deste projeto é utilizar o Apache Spark para processar um grande volume de eventos de dano extraídos de partidas competitivas de CS:GO, identificando padrões relacionados à eficiência das armas, ao desempenho dos jogadores e às vantagens estratégicas entre os lados Counter-Terrorist (CT) e Terrorist (T).

Este problema se enquadra no contexto de Big Data pelos seguintes motivos:

- O conjunto de dados possui aproximadamente seis milhões de registros.
- Os arquivos CSV ocupam mais de 1 GB de armazenamento.
- O conjunto completo é composto por diversas partes, totalizando vários gigabytes.
- O processamento envolve agregações e operações analíticas sobre milhões de linhas, tornando necessário o uso de processamento distribuído.

Dessa forma, foi escolhido o Apache Spark como framework de processamento distribuído, utilizando Docker para simular um cluster de execução.

2. Descrição do Conjunto de Dados

O conjunto de dados utilizado foi o **ESEA Master Damage Demos**, disponível publicamente na plataforma Kaggle.

Cada registro representa um evento de dano ocorrido durante uma partida competitiva de CS:GO.

Entre os principais atributos utilizados estão:

- Identificador da partida;
- Número da rodada;
- Arma utilizada;
- Categoria da arma;
- Dano causado à vida (HP);
- Dano causado à armadura;
- Rank do atacante;
- Rank da vítima;
- Lado do atacante (CT ou Terrorist);
- Lado da vítima;
- Indicador de bomba plantada;
- Região do corpo atingida.

O primeiro arquivo do conjunto de dados possui aproximadamente **5,99 milhões de registros**, ocupando cerca de **1,2 GB**. Nos experimentos finais foram utilizadas duas partes do dataset para aumentar a carga de processamento e permitir uma avaliação mais representativa do desempenho do Spark.

3. Arquitetura da Solução

Toda a aplicação foi desenvolvida utilizando Docker Compose para simular um ambiente distribuído.

A arquitetura é composta pelos seguintes componentes:

- Spark Master;
- Um ou mais Spark Workers;
- Spark Job (responsável pela execução da aplicação).

O Spark Job realiza a leitura dos arquivos CSV, executa os workloads implementados e salva os resultados obtidos em arquivos CSV. A quantidade de workers é definida durante a execução do benchmark, permitindo comparar o desempenho da aplicação utilizando diferentes níveis de paralelismo.

4. Workloads Implementados

Foram implementados três workloads com diferentes níveis de complexidade computacional:

Workload 1 – Eficiência das Armas

Agrupar todos os eventos de dano por arma e categoria da arma.

São calculadas as seguintes métricas:

- Quantidade de eventos;
- Dano médio causado à vida;
- Dano médio causado à armadura;
- Percentual de headshots.

Objetivo:

Identificar quais armas apresentam maior eficiência durante as partidas.

Workload 2 – Rank dos Jogadores versus Desempenho

Inicialmente são agregadas estatísticas individuais de cada jogador e, posteriormente, são calculadas métricas por rank.

As principais métricas calculadas são:

- Dano total causado;
- Dano médio;
- Quantidade de eventos.

Objetivo:

Verificar se jogadores de ranks mais elevados apresentam melhor desempenho em relação aos demais.

Workload 3 – Vantagem entre CT e Terrorist

Agrupar os eventos considerando:

- Lado do atacante;
- Lado da vítima;
- Situação da bomba (plantada ou não).

As métricas calculadas incluem:

- Quantidade de eventos;
- Dano médio;
- Dano total.

Objetivo:

Analisar se existe vantagem de dano para algum dos lados em diferentes situações da partida.

5. Ambiente Experimental

Os experimentos foram realizados utilizando:

- Sistema Operacional Linux;
- Docker;
- Docker Compose;
- Apache Spark 3.5.

Cada execução utilizou um cluster Docker configurado com diferentes quantidades de workers.

Foram realizados experimentos com:

- 1 Worker;
- 2 Workers;

- 4 Workers.

Cada configuração foi executada três vezes, permitindo calcular medidas estatísticas sobre os tempos de execução.

6. Metodologia Experimental

O objetivo dos experimentos foi analisar o impacto da quantidade de workers sobre o tempo de execução dos workloads.

Para cada configuração foram realizados os seguintes passos:

1. Inicialização do cluster Spark utilizando Docker Compose;
2. Leitura dos arquivos CSV;
3. Execução dos três workloads;
4. Registro do tempo total de execução de cada workload;
5. Repetição do experimento três vezes.

Ao final foram calculados:

- Média dos tempos de execução;
- Desvio padrão.

Essas métricas permitem avaliar tanto o desempenho médio quanto a estabilidade das execuções.

7. Resultados Experimentais

Tabela 1 – Tempo médio de execução dos workloads:

Workload	Workers	Média (s)	Desvio Padrão (s)
W1	1	2.06	0.12
W1	2	1.99	0.22
W1	4	1.89	0.12
W2	1	1.63	0.13
W2	2	1.61	0.16
W2	4	1.50	0.11
W3	1	1.36	0.19

Workload	Workers	Média (s)	Desvio Padrão (s)
W3	2	1.42	0.24
W3	4	1.37	0.21

8. Discussão dos Resultados

Os resultados obtidos mostram que o aumento do número de workers nem sempre resulta em uma redução proporcional do tempo de execução. Embora o Apache Spark seja uma plataforma de processamento distribuído, existe um custo associado à distribuição das tarefas entre os workers. Antes da execução das operações, o framework precisa particionar os dados, coordenar as tarefas e sincronizar os resultados, gerando um overhead que pode reduzir os ganhos do paralelismo, principalmente em workloads compostos por operações relativamente simples.

Os três workloads desenvolvidos utilizam predominantemente operações de agregação, como `groupBy`, `count`, `avg` e `sum`. Essas operações exigem que registros pertencentes ao mesmo grupo sejam reorganizados entre as partições durante o processamento, aumentando o custo de comunicação entre os workers. Além disso, observou-se uma variação considerável entre algumas execuções, refletida nos desvios padrão obtidos. Essa variação pode ser atribuída ao tempo de inicialização do Spark e da VM, ao overhead do Docker e à disponibilidade de recursos computacionais da máquina utilizada durante os experimentos.

Apesar dessas limitações, os experimentos demonstraram que o Apache Spark é capaz de processar milhões de registros de forma eficiente e escalável. À medida que o volume de dados cresce, espera-se que o custo fixo de inicialização e coordenação represente uma parcela menor do tempo total de execução, tornando os benefícios do processamento distribuído mais evidentes.

9. Repositório do Projeto

O projeto foi disponibilizado em um repositório público no GitHub contendo:

- Código-fonte da aplicação;
- Scripts de benchmark;

- Documentação do projeto;
- Slides de Apresentação;

Link: https://github.com/RATZogun/Final_Project_Big_Data.git